

Kompleksowa Dokumentacja Techniczna Aplikacji TipJar+

1.0 Wprowadzenie i Architektura Ogólna

1.1 Cel Dokumentu

Niniejszy dokument stanowi szczegółową specyfikację techniczną kluczowych modułów aplikacji TipJar+. Jego nadrzędnym celem jest służyć jako centralne i autorytatywne źródło wiedzy dla deweloperów, architektów oraz zespołów produktowych zaangażowanych w bieżące utrzymanie i dalszy rozwój systemu. Dokumentacja koncentruje się na precyzyjnym opisie architektury frontendu, strategii integracji z usługami backendowymi oraz mapowaniu kluczowych przepływów danych, które definiują podstawowe funkcjonalności platformy. Zapewnia to spójne rozumienie implementacji i ułatwia podejmowanie świadomych decyzji technologicznych.

1.2 Stos Technologiczny i Główne Komponenty

Architektura aplikacji TipJar+ opiera się na nowoczesnym i sprawdzonym stosie technologicznym, dobranym w celu zapewnienia wydajności, skalowalności i wysokiej jakości doświadczenia użytkownika. Poniżej przedstawiono kluczowe technologie pogrupowane według warstw systemu.

- **Frontend**
 - **Next.js (App Router):** Główny framework frontendowy, wykorzystujący architekturę App Router do budowy responsywnego i zoptymalizowanego pod kątem SEO interfejsu użytkownika. Zapewnia renderowanie po stronie serwera i klienta, co przekłada się na szybkość ładowania i płynność działania.
 - **Tailwind CSS:** System stylistyczny typu utility-first, używany do szybkiego i spójnego budowania interfejsu zgodnie z wytycznymi brandingu. Umożliwia tworzenie customowych motywów i zapewnia responsywność na różnych urządzeniach.
 - **wagmi:** Zestaw hooków React do interakcji z blockchainem Ethereum. W projekcie odpowiada za integrację z portfelami kryptowalutowymi, umożliwiając m.in. realizację procesu Sign-In with Ethereum (SIWE).

- **Backend**

- **NestJS:** Progresywny framework Node.js do budowy wydajnych i skalowalnych aplikacji po stronie serwera. Jego modułowa architektura jest wykorzystywana do tworzenia REST API obsługującego logikę biznesową, uwierzytelnianie oraz integracje z usługami zewnętrznymi.

- **Baza Danych i ORM**

- **Prisma:** Nowoczesny ORM (Object-Relational Mapping) dla Node.js, służący jako warstwa abstrakcji do komunikacji z bazą danych. Upraszcza operacje na danych i zarządzanie schematem, co jest bezpośrednio widoczne w implementacji kontrolera `CreatorOnboardingController`, który wstrzykuje i wykorzystuje `PrismaClient`.

- **Integracje Zewnętrzne**

- **Circle API:** Kluczowa usługa zewnętrzna, stanowiąca kręgosłup finansowy platformy. Wykorzystywana jest do tworzenia i zarządzania portfelami USDC w modelu *Developer Controlled Wallets (DCW)*, co pozwala na obsługę wpłat, wypłat i wewnętrznych transferów w sposób bezpieczny i zgodny z regulacjami, jednocześnie ukrywając złożoność technologii blockchain przed użytkownikiem końcowym.

2.0 UI Shell - Fundament Aplikacji Frontendowej

2.1 Koncepcja i Cele

Moduł UI Shell stanowi strategiczny fundament całej aplikacji frontendowej. Został zaprojektowany jako globalny szkielet interfejsu, który opakowuje wszystkie widoki i podstrony, zapewniając ich spójność strukturalną i wizualną. Jego głównymi celami są: unifikacja layoutu, implementacja stałych elementów nawigacyjnych (jak nagłówek i stopka), centralne zarządzanie globalnymi stylami oraz dostarczanie kluczowych kontekstów (providers) do całej drzewa komponentów. Dzięki temu UI Shell gwarantuje jednolite doświadczenie użytkownika i tworzy solidną, reużywalną podstawę pod dalszą rozbudowę aplikacji o nowe funkcjonalności.

2.2 Struktura `RootLayout` i Globalni Dostawcy (Providers)

Sercem UI Shell jest główny komponent `RootLayout`, zdefiniowany w pliku `src/app/layout.tsx`. Odpowiada on za renderowanie bazowej struktury

dokumentu HTML, w tym znaczników `<html>` i `<body>`. To w nim odbywa się integracja kluczowych elementów globalnych:

- **Import stylów:** Dołączany jest plik `globals.css`, który zawiera podstawowe style Tailwind CSS oraz niestandardowe definicje dla całej aplikacji.
- **Konfiguracja czcionek:** Przy użyciu `next/font` inicjowane są główne fonty aplikacji – `Mukta` (dla tekstu głównego) oraz `IBM_Plex_Sans` (dla elementów interfejsu). Są one udostępniane globalnie poprzez zmienne CSS.
- **Metadane SEO:** Definiowany jest domyślny tytuł strony oraz szablon, który pozwala na dynamiczne dostosowywanie tytułów na poszczególnych podstronach, co jest kluczowe dla optymalizacji pod kątem wyszukiwarek.

Kluczową rolę w architekturze pełni również komponent `<Providers>`, który działa jako wrapper dla całej aplikacji wewnątrz `RootLayout`. Jego zadaniem jest dostarczenie globalnych kontekstów, takich jak konfiguracja `wagmi`, co zapewnia dostęp do funkcji związanych z portfelem blockchain w każdym komponencie aplikacji bez konieczności manualnego przekazywania stanu.

2.3 Konfiguracja Stylów: Tailwind CSS i Branding

System stylistyczny aplikacji opiera się na konfiguracji Tailwind CSS, która została rozszerzona o elementy brandingu TipJar+, zapewniając wizualną spójność na całej platformie.

Paleta Kolorów

Kluczowe kolory brandowe zostały zdefiniowane w pliku `tailwind.config.ts`, co umożliwia ich semantyczne użycie w całej aplikacji.

Nazwa klasy	Kod Hex	Przeznaczenie
<code>brand-dark</code>	<code>#003737</code>	Ciemny turkus, główny kolor tła i komponentów.
<code>brand-gold</code>	<code>#FFD700</code>	Złoty, używany dla akcentów i kluczowych przycisków (CTA).
<code>brand-purple</code>	<code>#4D194D</code>	Fiolet, stosowany jako kolor pomocniczy w gradientach.
<code>text-primary</code>	<code>#DDE0DA</code>	Jasnoszary, podstawowy kolor tekstu na ciemnym tle.
<code>text-secondary</code>	<code>#BCC1B6</code>	Ciemniejszy odcień szarości dla tekstu pomocniczego.

Typografia

Aplikacja wykorzystuje dwie główne rodziny czcionek, zdefiniowane w konfiguracji Tailwind jako `font-sans` i `font-ui`:

- **Mukta Malar:** Używana jako podstawowa czcionka dla treści (`font-sans`), zapewniająca wysoką czytelność tekstu głównego.
- **IBM Plex Sans:** Stosowana dla elementów interfejsu użytkownika (`font-ui`), takich jak przyciski i nagłówki, nadając im nowoczesny i techniczny charakter.

Globalne Style

W pliku `globals.css` zdefiniowano globalny styl tła `bg-gradient-main`. Nadaje to interfejsowi głębię i nowoczesny wygląd, spójny z ciemnym motywem aplikacji. Jego precyzyjna definicja to `linear-gradient(135deg, hsla(180, 100%, 7%, 1) 0%, hsla(180, 100%, 8%, 1) 33%, hsla(195, 100%, 13%, 1) 60%, hsla(215, 100%, 11%, 1) 100%)`. UI Shell, poprzez centralizację tych definicji, stanowi gwarancję jednolitego i profesjonalnego wyglądu na wszystkich etapach interakcji użytkownika z platformą.

3.0 Moduł Uwierzytelniania i Zarządzanie Sesją

3.1 Architektura i Przepływ Danych

Moduł uwierzytelniania pełni strategiczną rolę w architekturze TipJar+, stanowiąc bramę do spersonalizowanych funkcji i zabezpieczając dostęp do wrażliwych danych, takich jak panel twórcy, historia transakcji czy portfel USDC. Jego nadrzędnym celem jest zapewnienie bezpiecznego i elastycznego procesu rejestracji, logowania oraz zarządzania sesją użytkownika. Realizacja tego celu opiera się na ścisłej integracji warstwy frontendowej, wykorzystującej bibliotekę NextAuth, z dedykowanym modułem Auth w backendzie opartym na NestJS, co umożliwia obsługę wielu strategii uwierzytelniania.

3.2 Uwierzytelnianie za pomocą E-maila i Hasła

Podstawową metodą uwierzytelniania jest tradycyjne logowanie za pomocą poświadczeń (e-mail i hasło). Proces ten jest obsługiwany przez komponent `LoginForm` po stronie frontendu, który komunikuje się z dedykowanymi endpointami REST API backendu:

- **Rejestracja:** Użytkownik, wypełniając formularz rejestracyjny, inicjuje żądanie `POST /api/v1/auth/register`. Backend NestJS jest odpowiedzialny za walidację danych, sprawdzenie unikalności adresu e-mail, utworzenie

nowego konta użytkownika w bazie danych oraz wygenerowanie tokenów JWT.

- **Logowanie:** Proces logowania polega na wysłaniu żądania `POST /api/v1/auth/login` z danymi uwierzytelniającymi. Backend weryfikuje poprawność poświadczeń i w przypadku sukcesu zwraca token JWT.

Token JWT jest następnie zarządzany po stronie klienta przez NextAuth, co umożliwia utrzymanie sesji i autoryzację kolejnych zapytań do chronionych zasobów API.

3.3 Strategie OAuth (Google, Twitch) i SIWE (Sign-In with Ethereum)

Aplikacja oferuje alternatywne, nowoczesne metody uwierzytelniania, aby sprostać oczekiwaniom różnych grup użytkowników.

- **OAuth (Google/Twitch):** Użytkownik może zalogować się za pomocą swojego konta Google lub Twitch. Proces ten inicjowany jest na frontendzie poprzez przekierowanie do odpowiedniego endpointu backendu (np. `/api/v1/auth/google`). Backend NestJS zarządza całym przepływem OAuth, komunikując się z dostawcą zewnętrznym, a po pomyślnej autoryzacji tworzy nowe konto w systemie TipJar+ lub loguje istniejącego użytkownika i zwraca token sesji.
- **SIWE (Sign-In with Ethereum):** Ta metoda umożliwia logowanie za pomocą portfela kryptowalutowego, co jest szczególnie istotne dla użytkowników Web3. Proces przebiega następująco:
 1. Frontend wysyła żądanie do backendu na endpoint `/api/v1/auth/siweNonce` w celu pobrania unikalnego, jednorazowego ciągu znaków (`nonce`). `Nonce` pełni rolę kryptograficznego wyzwania, które zapobiega atakom typu replay.
 2. Za pomocą biblioteki `wagmi`, frontend prosi użytkownika o podpisanie wiadomości zawierającej pobrany `nonce`.
 3. Podpisana wiadomość jest wysyłana do endpointu weryfikacyjnego `/api/v1/auth/siweVerify`.
 4. Backend weryfikuje poprawność podpisu i w przypadku sukcesu tworzy sesję dla użytkownika powiązanego z danym adresem portfela.

3.4 Zarządzanie Sesją i Pobieranie Danych Użytkownika

Po pomyślnym zalogowaniu, aplikacja zarządza sesją użytkownika. Kluczową rolę odgrywa tutaj endpoint `/api/v1/auth/me`. Frontend odpytuje ten zasób (przesyłając token JWT w nagłówku autoryzacyjnym) przy każdym odświeżeniu strony lub wejściu do aplikacji. Backend weryfikuje ważność tokenu i zwraca dane zalogowanego użytkownika (ID, rolę, e-mail itp.). Pozwala to na utrzymanie stanu zalogowania oraz dynamiczne renderowanie interfejsu w zależności od uprawnień użytkownika, stanowiąc fundament dla personalizowanych funkcji, takich jak panel twórcy czy dashboard fana.

4.0 Integracja z Circle API i Zarządzanie Portfelami USDC

4.1 Architektura Backendowa: Moduł `CircleService`

Integracja z Circle API stanowi fundament modelu biznesowego TipJar+, umożliwiając realizację kluczowych operacji finansowych na platformie. To właśnie ta integracja pozwala na tworzenie i zarządzanie portfelami w standardzie USDC w sposób bezpieczny, skalowalny i zgodny z regulacjami, jednocześnie w pełni abstrahując złożoność technologii blockchain od użytkownika końcowego. Logika tej integracji jest scentralizowana w dedykowanym module backendowym, co zapewnia spójność i bezpieczeństwo operacji.

4.2 Tworzenie Portfeli i Zarządzanie Saldem

Centralnym elementem architektury backendowej jest moduł `CircleModule` oraz zawarty w nim `CircleService` w aplikacji NestJS. Serwis ten jest odpowiedzialny za całą komunikację z Circle API. Kluczowy proces inicjowany jest w momencie rejestracji nowego twórcy:

1. Backend TipJar+ automatycznie wywołuje odpowiednią metodę Circle API w celu utworzenia dedykowanego portfela typu *Developer Controlled Wallet (DCW)*.
2. Po pomyślnym utworzeniu portfela, Circle API zwraca unikalne identyfikatory.
3. Backend zapisuje kluczowe dane, takie jak `circleWalletId` oraz `mainWalletAddress` (publiczny adres portfela), w bazie danych aplikacji, powiązując je z profilem danego użytkownika.

Gdy frontend potrzebuje wyświetlić aktualne saldo portfela, odwołuje się do dedykowanego endpointu w backendzie (np. `/api/v1/wallet`). Ten endpoint, wykorzystując `CircleService`, komunikuje się z Circle API, aby pobrać bieżące saldo i przekazać je do warstwy prezentacji.

4.3 Obsługa Transakcji: Napiwki i Wypłaty

System został zaprojektowany do obsługi dwóch głównych typów operacji finansowych, które są realizowane za pośrednictwem `CircleService`.

- **Transfery wewnętrzne (Napiwki):** Gdy fan decyduje się wesprzeć twórcę, operacja ta jest realizowana jako wewnętrzny transfer środków pomiędzy portfelami DCW w ekosystemie Circle. Taka architektura zapewnia, że transakcje są natychmiastowe i nie generują kosztów transakcyjnych (opłat *gas*), ponieważ stanowią off-chainowe aktualizacje w księdze głównej (ledger) w zamkniętym ekosystemie Circle, a nie transakcje na publicznym blockchainie.
- **Wypłaty (Payouts):** Twórca ma możliwość wypłaty zgromadzonych środków na zewnętrzny, osobisty adres portfela USDC. Proces ten inicjowany jest na frontendzie poprzez wysłanie żądania do endpointu `/api/v1/payouts`. Backend NestJS, po otrzymaniu takiego żądania, zleca transfer on-chain za pośrednictwem Circle API. W celu zapewnienia bezpieczeństwa, każde żądanie do Circle API jest wysyłane z unikalnym kluczem idempotencji. Jest to krytyczny wzorzec architektoniczny, który zapewnia odporność wywołań API do usług finansowych na błędy sieciowe, zapobiegając przypadkowym, zduplikowanym transakcjom.

Dzięki tej architekturze, integracja z Circle API stanowi solidny i bezpieczny kręgosłup finansowy dla całej platformy TipJar+.

5.0 Proces Onboardingu Twórcy

5.1 Koncepcja i Przebieg Użytkownika

Proces onboardingu ma kluczowe znaczenie strategiczne dla sukcesu twórcy na platformie TipJar+. Został zaprojektowany jako przyjazny, wieloetapowy kreator (wizard), którego celem jest płynne przeprowadzenie nowego użytkownika, rejestrującego się z rolą "Twórcy", przez wszystkie niezbędne kroki konfiguracyjne. Gwarantuje to, że profil twórcy jest kompletny i w pełni gotowy do przyjmowania wsparcia od fanów zaraz po zakończeniu procesu. Zastosowanie wizualnego wskaźnika postępu ma na celu redukcję obciążenia

poznawczego i motywowanie do ukończenia konfiguracji, podczas gdy personalizacja profilu poprzez ustawienie avatara znacząco zwiększa długoterminowe zaangażowanie użytkownika. Onboarding dla użytkowników z rolą "Fan" jest znacznie uproszczony i może zostać pominięty.

5.2 Struktura Kroków Onboardingu

Proces onboardingu został ustandaryzowany jako pięcioetapowy przepływ, który zbiera kluczowe informacje potrzebne do stworzenia w pełni funkcjonalnego profilu twórcy.

1. Krok 1: Tożsamość (Identity)

- **Cel:** Ustanowienie unikalnej tożsamości cyfrowej na platformie.
- **Działania:** Użytkownik ustawia swoją unikalną nazwę (`@username`), która będzie częścią jego publicznego adresu URL, oraz przesyła avatar i opcjonalnie zdjęcie w tle (cover photo) profilu.

2. Krok 2: Biografia i Media Społecznościowe (Bio & Social)

- **Cel:** Uzupełnienie profilu o informacje pozwalające fanom lepiej poznać twórcę.
- **Działania:** Twórca wprowadza swoją nazwę wyświetlaną (np. pseudonim artystyczny), pisze krótką biografię oraz dodaje linki do swoich profili na innych platformach społecznościowych, takich jak Twitch, YouTube, X (Twitter), Instagram czy strona internetowa.

3. Krok 3: Poziomy Wsparcia (Tiers)

- **Cel:** Konfiguracja modelu monetyzacji opartego na subskrypcjach.
- **Działania:** Twórca jest zachęcany do zdefiniowania co najmniej jednego poziomu wsparcia subskrypcyjnego, określając jego cenę i korzyści dla wspierających.

4. Krok 4: Płatności (Payments)

- **Cel:** Zapewnienie technicznej gotowości do przyjmowania i wypłacania środków.
- **Działania:** Na tym etapie następuje integracja z systemem płatności, co w praktyce oznacza finalne podłączenie i aktywację portfela USDC utworzonego przez Circle.

5. Krok 5: Publikacja (Publish)

- **Cel:** Ostateczne potwierdzenie i aktywacja publicznego profilu.
- **Działania:** Twórca dokonuje finalnego przeglądu wprowadzonych danych i jednym kliknięciem publikuje swój profil, czyniąc go widocznym dla wszystkich użytkowników platformy.

5.3 Implementacja Techniczna: Komponenty i Komunikacja z API

Realizacja techniczna procesu onboarding na frontendzie opiera się na komponencie `CreatorOnboardingWizard`. Jest on odpowiedzialny za zarządzanie ogólnym stanem kreatora, w tym śledzenie postępów i warunkowe renderowanie komponentów odpowiadających poszczególnym krokom. Każdy krok zawiera własny formularz i logikę, która po walidacji i zatwierdzeniu przez użytkownika komunikuje się z dedykowanym endpointem API w backendzie NestJS. Przykładowo, zapisanie tożsamości odbywa się poprzez wysłanie żądania `PATCH /api/v1/creator/onboarding/identity`, a dodanie poziomu wsparcia przez `POST /api/v1/creator/onboarding/tier`. Taki podział zapewnia modularność i łatwość w zarządzaniu procesem.

5.4 Zabezpieczenia (Guards) i Walidacja

Aby zagwarantować, że każdy twórca ukończy proces konfiguracji profilu, w aplikacji zaimplementowano mechanizm ochronny (tzw. "guard"). Działa on na poziomie serwera, za pomocą pliku `middleware.ts` w Next.js, i automatycznie przekierowuje zalogowanych twórców, którzy nie ukończyli jeszcze onboarding (`hasCompletedOnboarding === false`), z powrotem do kreatora przy próbie dostępu do innych sekcji panelu. Dodatkowo, na każdym etapie onboarding formularze są walidowane po stronie klienta (z użyciem bibliotek Zod i React Hook Form), aby zapewnić integralność i poprawność wprowadzanych danych, takich jak unikalność nazwy użytkownika czy format linków. Dzięki tym mechanizmom proces onboarding skutecznie przekształca nowo zarejestrowane, puste konto w pełni funkcjonalny i gotowy do działania profil twórcy.

6.0 Moduł Odkrywania Twórców (Explorer)

6.1 Cel i Ewolucja Funkcjonalności

Moduł Explorer stanowi publiczną witrynę platformy TipJar+, pełniąc kluczową rolę w ekosystemie poprzez łączenie fanów z twórcami. Jego głównym celem

jest umożliwienie użytkownikom (zarówno zalogowanym, jak i anonimowym) przeglądania, wyszukiwania i odkrywania profili twórców dostępnych w serwisie. Rozwój tej funkcjonalności był prowadzony w sposób iteracyjny, co pozwoliło na stopniowe budowanie jej wartości, zaczynając od prostego szkieletu, a kończąc na w pełni zintegrowanym i dynamicznym module.

6.2 Iteracyjny Rozwój (v1.1 - v1.4)

Ewolucja modułu Explorer przebiegała w zdefiniowanych etapach, z których każdy rozszerzał jego możliwości.

- **v1.1: Fundament i Statyczny Układ**
 - Na tym etapie stworzono podstawowy, statyczny layout strony z listą twórców. Zaimplementowano kluczowe komponenty UI, takie jak `CreatorsPage` (główny kontener) i `CreatorCard` (karta pojedynczego twórcy), wypełniając je danymi tymczasowymi (placeholderami).
- **v1.2 (Krok 1/3): Integracja z API i Wyszukiwanie**
 - W tej wersji dokonano integracji z backendem. Strona zaczęła dynamicznie pobierać listę rzeczywistych twórców z endpointu API `GET /api/v1/creators`. Dodano również podstawową funkcjonalność wyszukiwania, pozwalającą filtrować listę po nazwie twórcy.
- **v1.3 (Krok 2/3): Usprawnienia UX i Przygotowanie do Nawigacji**
 - Iteracja ta skupiła się na poprawie doświadczenia użytkownika. Wprowadzono mechanizm paginacji lub "infinite scroll" w celu efektywnego zarządzania dużą liczbą profili. Przygotowano również strukturę routingu (`/@{handle}`), aby umożliwić nawigację do indywidualnych stron profilowych.
- **v1.4 (Krok 3/3): Finalizacja i Pełna Integracja**
 - Ostatni etap polegał na pełnym zintegrowaniu modułu Explorer ze stronami profilowymi. Kliknięcie w kartę twórcy prowadzi teraz do dynamicznie generowanej strony `CreatorProfilePage`, która pobiera szczegółowe dane z endpointu `GET /api/v1/creator/{handle}`. Moduł został w pełni ukończony i stał się funkcjonalną bramą do wspierania twórców.

6.3 Kluczowe Komponenty i Struktura Danych

Architektura modułu Explorer opiera się na kilku kluczowych komponentach interfejsu użytkownika, które współpracują, aby dostarczyć spójne

doświadczenie.

- **CreatorsPage** : Jest to główny komponent-kontener dla całej strony "Odkrywaj". Odpowiada za logikę pobierania listy twórców z API, obsługę wyszukiwania i paginacji oraz renderowanie siatki lub listy kart twórców.
- **CreatorCard** : Reużywalny komponent, który wizualnie reprezentuje pojedynczego twórcę na liście. Wyświetla on kluczowe, skondensowane informacje, takie jak avatar, nazwa użytkownika i krótki opis. Pełni również funkcję nawigacyjną, linkując do pełnego profilu twórcy.
- **CreatorProfilePage** : Dynamicznie generowana strona publicznego profilu twórcy. Renderuje ona szczegółowe dane pobrane z backendu, w tym pełną biografię, linki do mediów społecznościowych oraz, co najważniejsze, zawiera osadzony komponent **TipForm**, umożliwiający fanom bezpośrednio wysłanie napiwku.

Razem, te komponenty tworzą spójny i intuicyjny przepływ, który skutecznie realizuje główny cel modułu Explorer – ułatwienie fanom odnalezienia i wsparcia swoich ulubionych twórców.

7.0 Testowanie i Zabezpieczenia Aplikacji

7.1 Strategia Testów End-to-End (E2E)

Aby zapewnić najwyższą jakość, stabilność i poprawność działania kluczowych przepływów biznesowych w aplikacji, wdrożono strategię automatycznych testów End-to-End (E2E). Testy te symulują rzeczywiste interakcje użytkownika z aplikacją w przeglądarce, weryfikując, czy poszczególne moduły poprawnie ze sobą współpracują od warstwy interfejsu aż po backend. Do realizacji i automatyzacji tych scenariuszy wykorzystano nowoczesny framework Playwright.

7.2 Główne Scenariusze Testowe

Zautomatyzowane testy E2E pokrywają najważniejsze i najbardziej krytyczne ścieżki użytkownika w aplikacji TipJar+.

- **Pełna Ścieżka Twórcy:**
 - **Symulowane działania:** Scenariusz symuluje rejestrację nowego konta twórcy, przechodzi przez wszystkie etapy wieloetapowego procesu onboardingu, wypełniając formularze i zapisując dane.

- **Oczekiwane rezultaty:** Test weryfikuje, czy po zakończeniu onboarding profil publiczny twórcy został poprawnie utworzony i jest dostępny, a użytkownik zostaje przekierowany do panelu Studio.
- **Interakcja Fana:**
 - **Symulowane działania:** Test symuluje fana, który wchodzi na stronę Explorer, używa wyszukiwarki do znalezienia konkretnego twórcy, przechodzi na jego stronę profilową, a następnie wypełnia formularz i wysyła napiwek.
 - **Oczekiwane rezultaty:** Test weryfikuje, czy saldo fana maleje, a statystyki twórcy (np. suma otrzymanych USDC, lista ostatnich transakcji w panelu Studio) są odpowiednio aktualizowane w interfejsie.
- **Weryfikacja Zabezpieczeń (Guardów):**
 - **Symulowane działania:** Scenariusz obejmuje próbę bezpośredniego dostępu do chronionych stron, takich jak panel twórcy (`/studio`), przez użytkownika, który nie jest zalogowany.
 - **Oczekiwane rezultaty:** Test potwierdza, że mechanizmy ochronne działają poprawnie, a użytkownik jest automatycznie przekierowywany na stronę logowania.
- **Skanowanie Placeholderów:**
 - **Symulowane działania:** Uruchamiany jest dedykowany skrypt, który skanuje całą aplikację w poszukiwaniu tymczasowych tekstów i etykiet, takich jak "Lorem ipsum" czy inne placeholderzy.
 - **Oczekiwane rezultaty:** Test kończy się sukcesem, jeśli w interfejsie nie zostaną znalezione żadne niezaimplementowane treści, co gwarantuje profesjonalny wygląd aplikacji w wersji produkcyjnej.

7.3 Mechanizmy Ochrony Tras (Routing Guards)

Kluczowym elementem zapewniającym bezpieczeństwo i prawidłowy przepływ użytkowników w aplikacji są mechanizmy ochrony tras, tzw. "guardy". Zostały one zaimplementowane w warstwie middleware w Next.js (plik `middleware.ts`). Ich zadaniem jest przechwytywanie żądań nawigacyjnych i weryfikacja uprawnień użytkownika przed udzieleniem dostępu do określonych sekcji aplikacji. Guardy działają w oparciu o dwa główne kryteria:

- **Stan uwierzytelnienia:** Sprawdzają, czy użytkownik jest zalogowany. Próba dostępu do stron chronionych (np. `/dashboard`, `/studio`) przez niezalogowanego użytkownika skutkuje natychmiastowym przekierowaniem do formularza logowania.
- **Status użytkownika:** Weryfikują dodatkowe atrybuty, takie jak status ukończenia procesu onboarding. Dzięki temu twórca, który nie skonfigurował jeszcze w pełni swojego profilu, jest automatycznie kierowany z powrotem do kreatora, co zapobiega dostępowi do nieprzygotowanych dla niego funkcji.

Te mechanizmy stanowią fundamentalną warstwę zabezpieczeń, chroniąc integralność danych i zapewniając, że użytkownicy poruszają się po aplikacji zgodnie z zaprojektowanym, logicznym przepływem.